



ClosingScript

This project may appeal to programmers who dislike CoffeeScript's reliance on significant indentation but otherwise appreciate the language for its simplicity and expressiveness.

ClosingScript addresses this issue by adding closing keywords to CoffeeScript syntax to terminate multi-line statements, making indentation no longer significant.

ClosingScript is a text preprocessor that regenerates significant indentation using these closing keywords before sending it to the official CoffeeScript compiler.

A closing keyword consists of the slash followed by the name of its matching statement.

List of closing keywords:

/if, /unless, /while, /until, /for, /switch, /loop, /try, /class, /end (function)

Advantages:

- Whitespace-insensitive syntax.
- Robust code structure.
- Clear visual termination of multi-line statements.
- Safe copy-pasting.
- Safe auto-formatting.

Disadvantages:

- Slightly more verbose syntax.

Syntax

The syntax is identical to CoffeeScript syntax except for the addition of a closing keyword for multi-line statements. The list below is not exhaustive and is provided to give a general idea.

One-liners for **if**, **unless**, **while**, **until** and **for** are unchanged and do not need closing keywords.

```
functionName = (parameters) -> | =>
  code
/end [optional following code]           # example of following code: /end, timeout
```

```
do ->
  code
/end
```

```
if condition
  code
[else if condition
  code]
[else
  code]
/if
```

```
while condition [guard clauses]
  code
/while
```

```
for iterable [guard clauses]
  code
/for
```

```
loop
  code
/loop
```

```
switch [variable]
when condition
  code
[when condition then code]
[else
  code]
/switch
```

```
class name
  code
/class
```

```
try
  code
[catch ...
  code]
[finally ...
  code]
/try
```

ClosingScript design decisions

Many different implementations of ClosingScript have been tried using different closing keywords and symbols. Some with more complex parsers and lexers which led to (much) bigger code but also with more possibilities of edge cases to creep in. Too many times, sneaky edge-cases were discovered late when everything seemed fine.

So the goal was reset to create a simple preprocessor without lexer with acceptable compromises listed below:

Multi-line block comment and heredoc:

Multi-line block comments and heredocs must be written on their own lines.

Examples:

```
###  
    multi-line  
    block comment  
###
```

```
” heredoc on  
multiple lines  
”
```

Variable assignment from a multi-line statement and heredoc:

The assigned variable must be put on the preceding line.

Examples:

```
variable =  
if condition  
    code  
/if
```

```
variable =  
” heredoc on  
multiple lines  
”
```

Usage

The preprocessor outputs to *stdout*. To get JavaScript, the official CoffeeScript compiler must be installed.

Any error message with a line number from the preprocessor or the compiler will match the same line in a ***ClosingScript*** source.

To convert and compile to JavaScript in one step:
closing *script.coffee* | **coffee** -s -c > *script.js*

To get a converted copy of a source:
closing *script.coffee* > *script.converted.coffee*

To get a formatted copy of a source (only reindented consistently):
closing -f *script.coffee* > *script.formatted.coffee*

Usage with the Geany code editor

In Linux, this command can be added to the Build menu of the Geany code editor. Its converts and compiles in one step, with errors falling back at guilty lines in the editor window, as well as error messages from the preprocessor and the compiler are sent to the ‘Compiler’ message window. This conveniently makes the whole process as a single compiling step to the user.

```
set -o pipefail; closing "%f" | coffee -s -c > "%e".js    2> >(sed 's\[stdin\]/%f/' >&2)
```